MA3632 Lecture 8 – Decision Trees

The models studied in Lectures 6 and 7 belong to the parametric family: they assume that the relationship between features and output can be captured by a fixed functional form (linear, logistic) whose parameters are estimated from data. Decision trees take a different approach. They make no assumption about the global form of f^*; instead, they recursively partition the feature space $\mathcal{X}$ into rectangular regions and assign a constant prediction to each region. The resulting hypothesis class, $\mathcal{H}_{\text{tree}}$, was introduced in Lecture 5 as one of the canonical examples of a non-parametric hypothesis class.

Decision trees have several properties that make them attractive in practice. They are interpretable: a fitted tree of moderate depth can be drawn and read by a non-specialist, making the prediction logic transparent. They handle mixed feature types (continuous and categorical) without preprocessing and require no feature scaling, unlike the distance-based methods of Lecture 5 and the regularised linear methods of Lectures 6 and 7. They can capture interactions between features and non-monotone relationships that linear models cannot represent. Their main weakness – high variance and instability – is addressed in Lecture 9 through ensemble methods.

1 The Hypothesis Class

Recall from Lecture 5 that $\mathcal{H}_{\text{tree},L}$ denotes the class of decision trees with at most L leaves. Each element of this class is a function $f\colon \mathcal{X} \to \mathcal{Y}$ defined by:

(i) a partition of $\mathcal{X}$ into at most L rectangular regions $R_1, \ldots, R_L$, where each region is defined by a sequence of axis-aligned splits of the form $x_j \leq \theta$ or $x_j > \theta$;

(ii) a constant prediction $c_\ell \in \mathcal{Y}$ assigned to each region R_ℓ.

For classification, c_ℓ is a class label; for regression, $c_\ell \in \mathbb{R}$.

A tree with L leaves has exactly $L-1$ internal nodes, each performing a binary split on one feature. The tree structure is a binary tree in which each internal node stores a splitting rule (j, θ) – split on feature j at threshold θ – and each leaf stores a prediction c_ℓ. The depth of the tree is the length of the longest path from root to leaf. A binary tree of depth d has at most 2^d leaves and $2^d - 1$ internal nodes.

Example 1.1. Consider a classification problem with two features $(x_1, x_2) \in \mathbb{R}^2$ and two classes. A tree with three leaves might implement the rule: *if $x_1 \leq 2$, predict class 0; else if $x_2 \leq 5$, predict class 1; else predict class 0.* This partitions $\mathbb{R}^2$ into three rectangles. The decision boundary consists of two line segments parallel to the coordinate axes, illustrating that decision tree boundaries are always axis-aligned – a fundamental structural constraint that distinguishes trees from the hyperplane boundaries of logistic regression.

Proposition 1.2. *For any bounded, continuous function $f^*\colon [0,1]^p \to \mathbb{R}$ and any $\varepsilon > 0$, there exists a decision tree T such that $\sup_{\mathbf{x}\in[0,1]^p} |f^*(\mathbf{x}) - T(\mathbf{x})| < \varepsilon$.*

This universal approximation property shows that decision trees are, in principle, capable of approximating any continuous function to arbitrary accuracy – provided the tree is deep enough. The argument is a direct consequence of uniform continuity on the compact domain $[0,1]^p$: choose $\delta > 0$ small enough that f^* varies by less than ε within any cube of side length δ, partition $[0,1]^p$ into a

grid of such cubes (realisable as a tree via repeated axis-aligned splits), and assign each leaf the value of f^* at any point in the corresponding cube. Continuity is essential here: a merely measurable function need not admit any such uniform approximation by a piecewise-constant function, since an arbitrarily fine partition may still fail to control the function's behaviour on badly discontinuous sets. The practical difficulty with the continuous case is that the number of leaves required may be exponential in the precision ε^{-1}, and finding the optimal tree of a given size is computationally intractable.

2 Recursive Partitioning

Finding the partition of $\mathcal{X}$ that globally minimises the empirical risk over $\mathcal{H}_{\text{tree},L}$ is NP-hard for $L \geq 3$. In practice, trees are built by a greedy top-down procedure known as *recursive binary splitting*.

Definition 2.1. Recursive binary splitting builds a tree as follows.

(i) Begin with the entire training set at a single root node.

(ii) At the current node containing training points $\mathcal{D}_t \subseteq \mathcal{D}$, consider all candidate splits (j, θ), where $j \in \{1, \ldots, p\}$ and θ ranges over the distinct observed values of feature j in $\mathcal{D}_t$. For each candidate split, compute the impurity reduction $\Delta(j, \theta)$ (defined in Section 3). Choose the split $(\hat{j}, \hat{\theta}) = \arg\max_{j,\theta} \Delta(j, \theta)$.

(iii) Partition $\mathcal{D}_t$ into left child $\mathcal{D}_L = \{(\mathbf{x}_i, y_i) \in \mathcal{D}_t : x_{i\hat{j}} \leq \hat{\theta}\}$ and right child $\mathcal{D}_R = \mathcal{D}_t \setminus \mathcal{D}_L$. Recurse on each child.

(iv) Stop when a termination criterion is met.

Remark 2.2. The greedy algorithm searches over all features and all observed values of each feature at each node. For a node with n_t observations and p features, the total number of candidate splits is at most $p(n_t - 1)$ (since a split at the k-th order statistic of feature j is equivalent to a split at any value in the interval between the k-th and $(k+1)$-th order statistics). The cost of finding the best split at one node is therefore $O(p\, n_t \log n_t)$, where the $\log n_t$ factor accounts for sorting each feature. The total cost of building a tree of depth d is $O(p\, n\, d \log n)$, which is manageable for moderate n, p, and d.

2.1 Categorical features

When feature j is categorical with K categories, a binary split partitions the K categories into two non-empty subsets $S \subset \{1, \ldots, K\}$ and its complement. There are $2^{K-1} - 1$ such partitions, which becomes combinatorially expensive for large K. For binary classification with Gini impurity, it can be shown that the optimal binary split on a categorical feature can always be found by ordering the categories by their class-1 frequency and splitting at a single threshold in this ordering – reducing the search to $K - 1$ candidates, the same as for a continuous feature. For multiclass problems or regression, this shortcut does not apply in general, and one may cap K or encode categories ordinally.

3 Impurity Measures

Let R be a node containing n_R training points, of which $n_{R,c}$ belong to class c, so $\hat{p}_c = n_{R,c}/n_R$ is the empirical class frequency at this node.

Definition 3.1. The **Gini impurity** of node R is

$$G(R) \;=\; \sum_{c=1}^{C} \hat{p}_c(1-\hat{p}_c) \;=\; 1 - \sum_{c=1}^{C} \hat{p}_c^{\,2}.$$

Definition 3.2. The **entropy** of node R is

$$H(R) \;=\; -\sum_{c=1}^{C} \hat{p}_c \log_2 \hat{p}_c,$$

with the convention $0 \log_2 0 = 0$.

Both measures are zero when the node is pure ($\hat{p}_c = 1$ for some c) and are maximised when all classes are equally represented. For binary classification ($C = 2$, writing $\hat{p} = \hat{p}_1$):

$$G(R) \;=\; 2\hat{p}(1-\hat{p}), \qquad H(R) \;=\; -\hat{p}\log_2 \hat{p} - (1-\hat{p})\log_2(1-\hat{p}).$$

Both are symmetric about $\hat{p} = 1/2$ and equal zero at $\hat{p} \in \{0, 1\}$. At $\hat{p} = 1/2$ the two measures take the values $G(1/2) = 1/2$ and $H(1/2) = 1$ bit, so the entropy is exactly twice the Gini impurity at this particular point; the ratio is not constant elsewhere, and grows without bound as $\hat{p} \to 0$ or $\hat{p} \to 1$.

Proposition 3.3. *For binary classification, $G(R) \le H(R) \ln 2$ for all $\hat{p} \in [0, 1]$.*

Proof. Since $\log_2(1/t) = -\log_2 t \ge (1-t)/\ln 2$ for all $t \in (0, 1]$ (this follows from $\ln t \le t - 1$, i.e. $-\ln t \ge 1 - t$, divided by $\ln 2$), applying this with $t = \hat{p}$ and $t = 1 - \hat{p}$ gives

$$H(R) \;=\; \hat{p}\log_2(1/\hat{p}) + (1-\hat{p})\log_2(1/(1-\hat{p})) \;\ge\; \hat{p}\cdot\frac{1-\hat{p}}{\ln 2} + (1-\hat{p})\cdot\frac{\hat{p}}{\ln 2} \;=\; \frac{2\hat{p}(1-\hat{p})}{\ln 2} \;=\; \frac{G(R)}{\ln 2}.$$

Multiplying both sides by $\ln 2$ gives the stated bound $G(R) \le H(R) \ln 2$. □

Remark 3.4. For regression trees, the impurity measure is the mean squared error within the node:

$$\mathrm{MSE}(R) \;=\; \frac{1}{n_R}\sum_{i \in R}(y_i - \bar{y}_R)^2, \qquad \bar{y}_R \;=\; \frac{1}{n_R}\sum_{i \in R} y_i.$$

Minimising $\mathrm{MSE}(R)$ within a leaf gives the sample mean $\bar{y}_R$ as the optimal constant prediction (proved in Section 6). For regression, the impurity reduction $\Delta(j, \theta)$ is the reduction in the parent-node MSE relative to the size-weighted average MSE of the two child nodes (equivalently, the reduction in within-node sum of squared errors up to a common scaling factor).

3.1 The splitting criterion

A split (j, θ) divides node R, containing $n_t = |R|$ training points, into left child $R_L = \{i \in R : x_{ij} \le \theta\}$ and right child $R_R = R \setminus R_L$. Write $n_L = |R_L|$ and $n_R = |R_R|$, so $n_L + n_R = n_t$. The **impurity reduction** is

$$\Delta(j, \theta) \;=\; I(R) \;-\; \frac{n_L}{n_t}\, I(R_L) \;-\; \frac{n_R}{n_t}\, I(R_R),$$

where I denotes the chosen impurity measure. The greedy algorithm selects $(\hat{j}, \hat{\theta}) = \arg\max_{j,\theta} \Delta(j, \theta)$.

Example 3.5. A binary classification node contains 10 observations: 6 of class 0 and 4 of class 1. The Gini impurity is $G(R) = 1 - (0.6^2 + 0.4^2) = 0.48$.
A candidate split produces R_L with 5 class-0 and 1 class-1 observation ($G(R_L) = 1-(25/36+1/36) = 10/36 \approx 0.278$) and R_R with 1 class-0 and 3 class-1 observation ($G(R_R) = 1 - (1/16 + 9/16) = 6/16 = 0.375$). The impurity reduction is

$$\Delta = 0.48 - \tfrac{6}{10}(0.278) - \tfrac{4}{10}(0.375) = 0.48 - 0.167 - 0.150 = 0.163.$$

Proposition 3.6. *The impurity reduction $\Delta(j, \theta) \geq 0$ for any split, with equality if and only if the class distribution is identical in R_L and R_R.*

Proof. Both Gini impurity and entropy are *strictly* concave functions of $(\hat{p}_1, \ldots, \hat{p}_C)$ on the probability simplex (apart from the usual boundary qualifications at pure nodes). Writing $\hat{\mathbf{p}}^{(L)}$ and $\hat{\mathbf{p}}^{(R)}$ for the class-frequency vectors of R_L and R_R, the class-frequency vector of the parent R is exactly the size-weighted average $(n_L/n_t)\hat{\mathbf{p}}^{(L)} + (n_R/n_t)\hat{\mathbf{p}}^{(R)}$. Jensen's inequality applied to this weighted average therefore gives $\Delta(j, \theta) \geq 0$, and *strict* concavity of I makes the inequality an equality precisely when $\hat{\mathbf{p}}^{(L)} = \hat{\mathbf{p}}^{(R)}$, i.e. when R_L and R_R have the same class distribution. □

This proposition confirms that each split either reduces impurity or leaves it unchanged. It does *not* follow that the greedy sequence of splits minimises the total impurity of the final tree: a split that reduces impurity slightly at the current node may enable much larger reductions at subsequent nodes, whereas the greedy optimum at each step may foreclose those later gains.

4 Stopping Criteria and Tree Depth

Recursive binary splitting continues until a stopping criterion is met. Standard criteria include:

Maximum depth $d_{\max}$: stop when the tree has reached depth $d_{\max}$. Controls complexity most directly.

Minimum samples per leaf $n_{\min}$: allow a split only if both resulting child nodes contain at least $n_{\min}$ training observations. This directly controls the minimum number of observations underlying any leaf prediction; it is a distinct condition from merely requiring the node being split to have at least $n_{\min}$ points, which would not by itself prevent a highly unbalanced split.

Minimum impurity reduction δ: do not split if no candidate split achieves $\Delta(j, \theta) \geq \delta$. Directly targets the stopping condition in terms of the objective.

These criteria are not mutually exclusive; in practice, all three are applied simultaneously, with a split attempted only if all criteria allow it.

The bias–variance trade-off governs the choice of stopping criterion. When the training observations can be separated by the available features, a sufficiently deep unconstrained tree can place individual observations (or pure groups of them) in separate leaves and achieve zero training classification error; this need not hold when, for instance, two observations share identical feature values but carry conflicting labels, in which case the leaf containing them cannot be made pure by any further splitting. Away from such degeneracies, an unconstrained tree grown to full depth is the tree analogue of k-NN with $k = 1$: it memorises $\mathcal{D}$ almost entirely. As shown in Lecture 5, such a model has very low bias (it can represent almost any labelling of the training data) but very high variance (the prediction at a query point depends on a very small, specific subset of training points, and this can change dramatically with small perturbations to $\mathcal{D}$).

Restricting the depth introduces bias: the partition is too coarse to capture fine structure in f^*, so some leaves will contain points from multiple true classes and the prediction will be the plurality class rather than the true class. But it reduces variance substantially: the prediction at any query point is now determined by a group of training points, and the group membership is more stable across different realisations of $\mathcal{D}$.

Proposition 4.1. *For a regression tree with mean squared error as the impurity measure and at least one training point in each leaf, the prediction c_ℓ that minimises the within-leaf empirical risk is the sample mean $\bar{y}_\ell = n_\ell^{-1} \sum_{i \in \ell} y_i$. For a classification tree under 0–1 loss, the optimal leaf prediction is the plurality class $c_\ell = \arg\max_c n_{\ell,c}$.*

Proof. For regression: $\sum_{i \in \ell}(y_i - c_\ell)^2$ is minimised by $c_\ell = n_\ell^{-1} \sum_{i \in \ell} y_i$ (differentiate and set to zero). For classification: $\sum_{i \in \ell} \mathbf{1}[y_i \neq c_\ell] = n_\ell - n_{\ell,c_\ell}$ is minimised by maximising n_{ℓ,c_ℓ}, i.e. by choosing the plurality class. □

5 Pruning

An alternative to stopping criteria is to grow a large tree and then *prune* it. Pre-pruning (stopping early) is simple but myopic: it may stop a split that looks uninformative in isolation but would have enabled large impurity reductions one level deeper. Post-pruning avoids this by observing the full tree before deciding which branches to remove.

Definition 5.1. Let $T_{\max}$ be the tree grown to full depth. For $\alpha \geq 0$, define the **cost-complexity** of a subtree $T \subseteq T_{\max}$ as

$$R_\alpha(T) \;=\; R(T) \;+\; \alpha\,|T|,$$

where $R(T) = n^{-1} \sum_{\ell=1}^{|T|} \sum_{i \in \ell} I(y_i, c_\ell)$ is the total training impurity (weighted sum of leaf impurities) and $|T|$ is the number of leaves. **Cost-complexity pruning** finds, for each α, the subtree $T_\alpha = \arg\min_{T \subseteq T_{\max}} R_\alpha(T)$.

The parameter α penalises each additional leaf. When $\alpha = 0$, $T_0 = T_{\max}$ (more leaves always reduce or maintain $R(T)$, so the full tree is preferred). As α increases, the penalty makes additional leaves less attractive and the optimal subtree shrinks. This generates a finite sequence of nested subtrees:

$$T_{\max} = T_0 \;\supseteq\; T_{\alpha_1} \;\supseteq\; T_{\alpha_2} \;\supseteq\; \cdots \;\supseteq\; T_\infty \;=\; \{\text{root}\},$$

where $0 < \alpha_1 < \alpha_2 < \cdots$ are the values at which the optimal subtree changes. Cross-validation selects the best α from this sequence.

Theorem 5.2 (Weakest link pruning)**.** *The sequence of optimal subtrees $T_0 \supseteq T_{\alpha_1} \supseteq \cdots$ can be computed in polynomial time by iteratively collapsing the internal node t^* whose removal causes the smallest increase in $R(T)$ per leaf removed:*

$$t^* \;=\; \arg\min_{t \,\in\, \text{internal}(T)} \frac{R(T_t) - R(T)}{|T| - |T_t|},$$

where T_t denotes the tree T with the subtree rooted at t collapsed to a leaf. The threshold at which t^* is collapsed is $\alpha^* = (R(T_{t^*}) - R(T))/(|T| - |T_{t^*}|)$.

The ratio in Theorem 5.2 has a natural interpretation: it is the reduction in $R(T)$ per leaf gained by keeping the subtree rooted at t rather than collapsing it. When this ratio falls below α, collapsing is cheaper than keeping the subtree.

Example 5.3. Suppose a tree has five leaves and total training impurity $R(T) = 0.08$. An internal node t has a subtree with two leaves contributing $R_{\text{sub}} = 0.10$ to the total impurity (i.e. collapsing t to a leaf would change R from 0.08 to $0.08 - 0.10 + R_t$ where R_t is the impurity of the collapsed leaf, say $R_t = 0.13$). Collapsing t changes the total impurity by $+0.03$ and reduces the leaf count by 1 (from two sub-leaves to one). The weakest-link ratio for node t is $0.03/1 = 0.03$. If no other internal node has a smaller ratio, t^* is the first node collapsed, at $\alpha^* = 0.03$. For any $\alpha > 0.03$, the pruned tree (with t collapsed) is preferred to the full tree.

6 Predictions and Leaf Assignment

Given a fitted tree, predicting the output for a new observation $\mathbf{x}$ requires traversing the tree from root to leaf. At each internal node, one evaluates the splitting rule: go left if $x_j \leq \theta$, go right otherwise. This traversal costs $O(d)$ where d is the depth of the tree. The leaf prediction c_ℓ is returned.

For classification, one can return either the plurality class or the empirical class distribution $(\hat{p}_1, \ldots, \hat{p}_C)$ at the leaf. The empirical distribution allows computation of metrics such as AUC (Lecture 7) and is required by ensemble methods. However, these probabilities are often poorly calibrated: a leaf containing only class-1 observations returns $\hat{p}_1 = 1$ regardless of how few observations it contains, overstating confidence. Pruning and minimum leaf size requirements mitigate this by ensuring each leaf contains a reasonable number of training points.

It is worth noting that the decision boundary of a classification tree is determined entirely by the training data: unlike logistic regression, where the boundary is a hyperplane whose position is determined by all n training points simultaneously, a tree's boundary is a union of axis-aligned line segments, each determined locally by a single split. This makes trees more flexible but also more sensitive to individual training points near the boundary.

7 Feature Importance

A decision tree induces a natural measure of the importance of each feature. Features used near the top of the tree (close to the root) govern predictions for more training points and are therefore more influential than features used only in small subtrees near the leaves.

Definition 7.1. The **mean decrease in impurity** (MDI) of feature j in a fitted tree T is

$$\text{MDI}(j) \;=\; \sum_{\substack{t \in \text{internal}(T) \\ j_t = j}} \frac{n_t}{n}\, \Delta(j, \theta_t),$$

where the sum is over all internal nodes t at which feature j is used to split, n_t is the number of training points reaching node t, n is the total training set size, and $\Delta(j, \theta_t)$ is the impurity reduction at node t. The MDI values are typically normalised to sum to one across all features.

The MDI of feature j accumulates the weighted impurity reductions it achieves across all the nodes where it is selected. A feature that appears only in small, deep nodes contributes little; a feature that is selected at the root (where $n_t = n$ and Δ is typically large) contributes substantially.

Proposition 7.2. *Let I_{root} denote the impurity of the root node and let I_{leaves} denote the weighted average impurity of the leaves. Then*

$$\sum_{j=1}^{p} \text{MDI}(j) \;=\; I_{\text{root}} - I_{\text{leaves}}.$$

Proof. Each internal node t contributes $\frac{n_t}{n}\Delta(j_t, \theta_t)$ to $\mathrm{MDI}(j_t)$. Summing over all internal nodes, and using the telescoping structure of the tree (each node's impurity reduction accounts for the gap between its impurity and the weighted average of its children's impurities), the total equals the reduction in impurity from the root to the leaves. □

Remark 7.3. MDI has a well-documented bias: features with many distinct values (high-cardinality continuous features or categorical features with many categories) have more candidate split points at each node and are therefore more likely to be selected by the greedy algorithm, even if they are not genuinely more predictive than a low-cardinality feature. This bias is purely an artefact of the splitting procedure and not a reflection of true feature relevance.
A bias-free alternative is **permutation importance**: after fitting the model and evaluating it on a held-out test set, randomly permute the values of feature j across the test observations (breaking the association between feature j and the target) and re-evaluate the model. The permutation importance of feature j is the increase in test error caused by this permutation. Features that are genuinely predictive will cause a large increase in error when permuted; irrelevant features will cause little change. Permutation importance does not suffer from the same high-cardinality bias as MDI, although it can itself be misleading when features are strongly correlated, and is slower to compute and requires a held-out set. Both measures are examined in the Workshop.

8 Bias–Variance Analysis

The bias–variance decomposition of Lecture 5 applies directly to trees. Define the *effective complexity* of a tree by its number of leaves L (equivalently its depth d for balanced trees).

As L increases from 1 (root-only tree, predicting the majority class everywhere) to n (approximately one point per leaf), the tree becomes more flexible and can generally reduce approximation bias: with more leaves, the partition is finer and the prediction can typically track f^* more closely. The variance increases: with more leaves, each leaf contains fewer training points, and the leaf prediction is more sensitive to which particular points fell in that leaf. The total expected error is typically U-shaped in L, and the optimal L is found by cross-validation.

A specific and important form of the variance is *instability*: a small change in the training data (replacing one point, correcting one label, or adding one observation) can cause the *first* split of the tree to change, which cascades into an entirely different tree structure below it. This makes decision trees one of the most unstable learners in common use. Contrast this with a well-conditioned linear regression problem, where small perturbations to the data typically produce correspondingly small changes in the fitted coefficients.

The instability of trees is not merely an inconvenience; it is the key property that makes ensemble methods effective. If one trains B trees on B bootstrap resamples of $\mathcal{D}$ and averages their predictions, each tree has high variance, but the average can have substantially lower variance. The bootstrap trees are not independent – their predictions are generally correlated, since they are trained from resamples of the same underlying dataset – but averaging still reduces the uncorrelated component of their variance. For a collection of B identically distributed estimators with variance σ^2 and pairwise correlation ρ, the variance of their average is

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\right) \;=\; \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

As $B \to \infty$, this converges to $\rho\sigma^2$, the variance that remains due to correlation between trees. Random Forests (Lecture 9) reduce ρ further by introducing additional randomness in the splitting step.

9 Exercises

Exercise 9.1. A classification node contains 12 training observations: 8 of class A and 4 of class B. A proposed split produces a left child with 7 class-A and 1 class-B observation, and a right child with 1 class-A and 3 class-B observation.
Compute the Gini impurity of the parent node and each child. Compute the impurity reduction Δ. Compute also the entropy of each node and the entropy-based impurity reduction. State the plurality-class prediction at each node.

Solution. **Gini impurity.** Parent: $\hat{p}_A = 2/3$, $\hat{p}_B = 1/3$. $G(\text{parent}) = 1-(4/9+1/9) = 4/9 \approx 0.444$.
Left child ($n_L = 8$): $\hat{p}_A = 7/8$, $\hat{p}_B = 1/8$. $G(R_L) = 1 - (49/64 + 1/64) = 14/64 = 7/32 \approx 0.219$.
Right child ($n_R = 4$): $\hat{p}_A = 1/4$, $\hat{p}_B = 3/4$. $G(R_R) = 1 - (1/16 + 9/16) = 6/16 = 3/8 = 0.375$.
Gini impurity reduction: $\Delta_G = 4/9 - (8/12)(7/32) - (4/12)(3/8) = 4/9 - 7/48 - 1/8$. Common denominator 144: $\Delta_G = 64/144 - 21/144 - 18/144 = 25/144 \approx 0.174$.
Entropy. Parent: $H = -(2/3)\log_2(2/3) - (1/3)\log_2(1/3) = (2/3)\log_2(3/2) + (1/3)\log_2 3$. Since $\log_2(3/2) \approx 0.585$ and $\log_2 3 \approx 1.585$: $H(\text{parent}) \approx 0.390 + 0.528 = 0.918$ bits.
Left child: $H(R_L) = -(7/8)\log_2(7/8) - (1/8)\log_2(1/8)$. $\log_2(8/7) \approx 0.193$, $\log_2 8 = 3$: $H(R_L) \approx (7/8)(0.193) + (1/8)(3) = 0.169 + 0.375 = 0.544$ bits.
Right child: $H(R_R) = -(1/4)\log_2(1/4) - (3/4)\log_2(3/4)$. $\log_2 4 = 2$, $\log_2(4/3) \approx 0.415$: $H(R_R) \approx (1/4)(2) + (3/4)(0.415) = 0.500 + 0.311 = 0.811$ bits.
Entropy impurity reduction: $\Delta_H = 0.918 - (8/12)(0.544) - (4/12)(0.811) = 0.918 - 0.363 - 0.270 = 0.285$ bits.
Plurality predictions. Parent: class A ($8 > 4$). Left child: class A ($7 > 1$). Right child: class B ($3 > 1$). □

Exercise 9.2. Verify Proposition 3.3 numerically by computing $G(\hat{p})$ and $H(\hat{p})\ln 2$ for $\hat{p} \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and confirming that $G(\hat{p}) \le H(\hat{p})\ln 2$ in each case. Also verify that both measures equal zero at $\hat{p} = 0$ and $\hat{p} = 1$.

Solution. For binary classification, $G(\hat{p}) = 2\hat{p}(1-\hat{p})$ and $H(\hat{p}) = -\hat{p}\log_2\hat{p} - (1-\hat{p})\log_2(1-\hat{p})$.

$\hat{p}$	$G(\hat{p})$	$H(\hat{p})$ (bits)	$H(\hat{p})\ln 2$	$G \le H\ln 2$?
0.0	0.000	0.000	0.000	Yes
0.1	0.180	0.469	0.325	Yes
0.3	0.420	0.881	0.611	Yes
0.5	0.500	1.000	0.693	Yes
0.7	0.420	0.881	0.611	Yes
0.9	0.180	0.469	0.325	Yes
1.0	0.000	0.000	0.000	Yes

The inequality $G(\hat{p}) \le H(\hat{p})\ln 2$ holds at every value and is strict for every $\hat{p} \in (0,1)$. At $\hat{p} = 1/2$, $G(1/2) = 0.5$ while $H(1/2)\ln 2 = \ln 2 \approx 0.693$. Both measures vanish at $\hat{p} \in \{0,1\}$, where equality holds. □

Exercise 9.3. Explain in terms of the bias–variance decomposition why a decision tree grown to full depth achieves zero (or near-zero) training error but is likely to have high generalisation error when training labels are noisy. How does restricting the maximum depth address each of bias and variance, and what does this imply about using training error to select the depth?
A tree of depth $d = 3$ is fitted to a dataset and achieves a training error of 0.08 and a cross-validation error of 0.15. A tree of depth $d = 5$ achieves a training error of 0.03 and a cross-validation error of 0.14. A tree of depth $d = 7$ achieves a training error of 0.01 and a cross-validation error of 0.19. Which depth would you select and why? What does the large gap between training and cross-validation error at $d = 7$ indicate?

Solution. When the training observations can be separated by the available features, a fully grown tree can place individual training points, or pure groups of observations, into separate leaves and predict their labels correctly, so the training error is zero, or very close to zero in the rare case where identical feature vectors carry conflicting labels. If labels contain noise ε_i, the tree memorises the noise as well as the signal. Different realisations of $\mathcal{D}$ produce completely different leaf assignments for query points near the training observations, so the variance of the prediction is high. The bias is low (the tree can represent almost any labelling of $\mathcal{D}$) but the noise memorisation dominates the generalisation error.
Restricting the depth forces each leaf to contain multiple training points. The leaf prediction averages their labels, partially cancelling noise (variance falls). The cost is increased bias: the partition is too coarse to track fine structure in f^*, so some leaves aggregate points from different true classes and the plurality prediction is further from f^*.
Training error decreases (weakly) monotonically with depth and reaches zero, or close to it, for a fully grown tree. It therefore carries little information about generalisation and cannot be used on its own to select the depth. Cross-validation estimates the generalisation error using held-out folds and typically produces a U-shaped curve that can be used to select the depth.
For the numerical comparison: select $d = 5$. It achieves the lowest cross-validation error (0.14) among the three candidates. The large gap between training error (0.01) and cross-validation error (0.19) at $d = 7$ indicates substantial overfitting: the tree has memorised the training data to an extent that does not transfer to new observations. The gap is the *generalisation gap* $R(\hat{f}) - \hat{R}(\hat{f})$, which grows as the model becomes more complex. □

Exercise 9.4. Consider the cost-complexity pruning criterion $R_\alpha(T) = R(T) + \alpha|T|$.

(a) Show that for $\alpha = 0$, the full tree $T_{\max}$ minimises $R_\alpha(T)$ among all subtrees.

(b) Show that for sufficiently large α, the root-only tree (one leaf) minimises $R_\alpha(T)$.

(c) Three candidate subtrees have the following properties: T_1: 5 leaves, $R(T_1) = 0.10$; T_2: 3 leaves, $R(T_2) = 0.14$; T_3: 1 leaf, $R(T_3) = 0.22$. Find the values of α at which the optimal subtree changes (the *crossover* values) and state which tree is optimal in each interval.

Solution. (a) When $\alpha = 0$, $R_0(T) = R(T)$. Every split at an internal node either reduces or maintains the total impurity (Proposition 3.6), so $R(T_{\max}) \leq R(T)$ for any subtree T. Hence $T_{\max}$ minimises R_0.
(b) Let T_1 be the root-only tree with $|T_1| = 1$ and let T be any tree with $|T| \geq 2$ leaves. Then $R_\alpha(T) - R_\alpha(T_1) = (R(T) - R(T_1)) + \alpha(|T| - 1)$. As $\alpha \to \infty$, the second term dominates (since $|T| - 1 \geq 1$), so $R_\alpha(T) > R_\alpha(T_1)$ for all $T \neq T_1$. Therefore T_1 (the root-only tree) is optimal for all sufficiently large α.
(c) The penalised costs are: $R_\alpha(T_1) = 0.10 + 5\alpha$, $R_\alpha(T_2) = 0.14 + 3\alpha$, $R_\alpha(T_3) = 0.22 + \alpha$.

Crossover between T_1 and T_2: $0.10 + 5\alpha = 0.14 + 3\alpha \Rightarrow 2\alpha = 0.04 \Rightarrow \alpha = 0.02$.
Crossover between T_2 and T_3: $0.14 + 3\alpha = 0.22 + \alpha \Rightarrow 2\alpha = 0.08 \Rightarrow \alpha = 0.04$.
Check that T_1 and T_3 do not directly cross before T_2 intervenes: $0.10 + 5\alpha = 0.22 + \alpha \Rightarrow \alpha = 0.03$, which lies between 0.02 and 0.04, so T_2 is indeed the intermediate-optimal tree.
Optimal tree by interval: $[0, 0.02)$: T_1 (five leaves); at $\alpha = 0.02$: T_1 and T_2 tie; $(0.02, 0.04)$: T_2 (three leaves); at $\alpha = 0.04$: T_2 and T_3 tie; $(0.04, \infty)$: T_3 (root only). □

Exercise 9.5. Let T be a classification tree fitted on a training set $\mathcal{D}$ of $n = 100$ points with $p = 4$ features. The tree has depth 3 and the following structure at its internal nodes (given as feature index, impurity reduction, and number of training points reaching the node):

Node	Depth	Feature	Δ	n_t
Root	0	x_1	0.180	100
Left child of root	1	x_3	0.090	60
Right child of root	1	x_1	0.060	40
Left-left grandchild	2	x_2	0.040	35
Left-right grandchild	2	x_4	0.020	25
Right-left grandchild	2	x_3	0.050	22
Right-right grandchild	2	x_1	0.030	18

Compute the unnormalised MDI for each of the four features x_1, x_2, x_3, x_4. Verify that the sum of all MDI values equals the total impurity reduction from root to leaves (Proposition 7.2). Normalise the MDI values to sum to one and rank the features by importance.

Solution. Recall $\mathrm{MDI}(j) = \sum_{t:j_t=j} (n_t/n)\,\Delta_t$ with $n = 100$.
Feature x_1: appears at the root ($n_t = 100$, $\Delta = 0.180$), the right child of root ($n_t = 40$, $\Delta = 0.060$), and the right-right grandchild ($n_t = 18$, $\Delta = 0.030$).

$$\mathrm{MDI}(x_1) = \frac{100}{100}(0.180) + \frac{40}{100}(0.060) + \frac{18}{100}(0.030) = 0.180 + 0.024 + 0.0054 = 0.2094.$$

Feature x_2: appears at the left-left grandchild ($n_t = 35$, $\Delta = 0.040$).

$$\mathrm{MDI}(x_2) = \frac{35}{100}(0.040) = 0.0140.$$

Feature x_3: appears at the left child of root ($n_t = 60$, $\Delta = 0.090$) and the right-left grandchild ($n_t = 22$, $\Delta = 0.050$).

$$\mathrm{MDI}(x_3) = \frac{60}{100}(0.090) + \frac{22}{100}(0.050) = 0.054 + 0.011 = 0.0650.$$

Feature x_4: appears at the left-right grandchild ($n_t = 25$, $\Delta = 0.020$).

$$\mathrm{MDI}(x_4) = \frac{25}{100}(0.020) = 0.0050.$$

Total MDI $= 0.2094 + 0.0140 + 0.0650 + 0.0050 = 0.2934$.
Verification: the total impurity reduction from root to leaves equals the sum of all weighted impurity reductions at internal nodes: $(100/100)(0.180) + (60/100)(0.090) + (40/100)(0.060) + (35/100)(0.040) + (25/100)(0.020) + (22/100)(0.050) + (18/100)(0.030) = 0.180 + 0.054 + 0.024 + 0.014 + 0.005 + 0.011 + 0.0054 = 0.2934$.
Normalised MDI (divide by 0.2934): x_1: $0.2094/0.2934 \approx 0.714$; x_3: $0.0650/0.2934 \approx 0.222$; x_2: $0.0140/0.2934 \approx 0.048$; x_4: $0.0050/0.2934 \approx 0.017$.
Ranking: $x_1 \gg x_3 > x_2 > x_4$. Feature x_1 dominates because it appears at the root (governing all 100 training points) with a large impurity reduction, and also appears twice deeper in the tree. □

Further Reading

Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth. The original monograph introducing CART, recursive binary splitting with Gini impurity, and cost-complexity pruning, all covered in this lecture.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106. The original ID3 algorithm using entropy and information gain as the splitting criterion, complementing the Gini-based CART approach.

Lindholm, A., Wahlström, N., Lindsten, F., and Schön, T. B. (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press. Introduces decision trees alongside k-nearest neighbours as a first, non-parametric approach to supervised learning.

James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Chapter 8 covers tree-based methods in detail, including cost-complexity pruning and both MDI and permutation-based feature importance.

References

Breiman, L., Friedman, J.H., Olshen, R.A. and Stone, C.J., 1984. *Classification and Regression Trees*. Wadsworth.

James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J., 2023. *An Introduction to Statistical Learning with Applications in Python*. Springer.

Lindholm, A., Wahlström, N., Lindsten, F. and Schön, T.B., 2022. *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.

Quinlan, J.R., 1986. Induction of decision trees. *Machine Learning, 1*(1), pp.81–106.